

Embedded Linux & System Software Interview Handbook

Chapter 5 - Linux Device Drivers (Part 4)

Topics: Complete Character Driver Workflow, file_operations, ioctl(), Testing and Debugging

Character Driver Architecture

Applications communicate with a character driver using open(), read(), write() and ioctl(). The Virtual File System (VFS) routes these system calls to the callbacks registered in struct file_operations.

Application -> System Call -> VFS -> file_operations -> Driver -> Hardware

Driver Development Steps

Step	Activity
1	Allocate device numbers
2	Initialize cdev
3	Register cdev
4	Create device class
5	Create /dev node
6	Implement callbacks
7	Unload and clean up

Important Callbacks

open(), release(), read(), write() and unlocked_ioctl() are the most common callbacks implemented by a character driver.

Sample file_operations

```
static const struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = drv_open,
    .read = drv_read,
    .write = drv_write,
    .unlocked_ioctl = drv_ioctl,
    .release = drv_release,
};
```

Safe Data Transfer

Use copy_to_user() and copy_from_user() for transferring data between kernel space and user space. Never access user pointers directly inside the kernel.

Testing Workflow

Build the module, load it using insmod/modprobe, verify with lsmod, inspect printk() output using dmesg, confirm the /dev node exists, then test using a user-space application.

Common Bugs

Memory leaks, missing cleanup, incorrect major/minor numbers, invalid ioctl handling, race conditions and missing error checks are common issues during driver development.

Interview Questions

1. Why use copy_to_user()?
2. Explain unlocked_ioctl().
3. How does VFS invoke driver callbacks?
4. What happens when open() fails?
5. How do you debug a character driver that does not create a /dev node?

Key Takeaways

A robust character driver requires correct registration, safe user-kernel communication, careful cleanup and systematic debugging using kernel logs and user-space tests.